

PROJECTION

1. Создаем таблицу sales

```
CREATE TABLE sales (  
  id UInt32,  
  product_id UInt32,  
  quantity UInt32,  
  price Float32,  
  sale_date DateTime  
) ENGINE = MergeTree()  
ORDER BY (product_id, sale_date);
```

```
CREATE TABLE sales  
(  
  `id` UInt32,  
  `product_id` UInt32,  
  `quantity` UInt32,  
  `price` Float32,  
  `sale_date` DateTime  
)  
ENGINE = MergeTree  
ORDER BY (product_id, sale_date)
```

Query id: 86deca4f-0f68-4d24-b118-e3a879437627

Ok.

0 rows in set. Elapsed: 0.034 sec.

2. Добавляем данные

```
INSERT INTO sales  
SELECT  
  number + 1 AS id,  
  rand() % 1000 + 1 AS product_id,  
  rand() % 10 + 1 AS quantity,  
  (rand() % 1000 + 1) / 10.0 AS price,  
  now() - toIntervalDay(rand() % 365) AS sale_date  
FROM numbers(100000);
```

```
INSERT INTO sales SELECT  
  number + 1 AS id,  
  (rand() % 1000) + 1 AS product_id,  
  (rand() % 10) + 1 AS quantity,  
  ((rand() % 1000) + 1) / 10. AS price,  
  now() - toIntervalDay(rand() % 365) AS sale_date  
FROM numbers(100000)
```

Query id: a1d7dcf3-b169-4d6f-9f11-ee3180dc4672

→ Progress: 100.00 thousand rows, 800.00 KB (5.05 million rows/s., 40.39 MB/s.) ↘ Progress: 100.00 thousand rows, 800.00 KB (5.05 million rows/s., 40.39 MB/s.) Ok.

0 rows in set. Elapsed: 0.020 sec. Processed 100.00 thousand rows, 800.00 KB (5.05 million rows/s., 40.39 MB/s.)

Peak memory usage: 8.00 MiB.

3. Создаем проекцию

```
ALTER TABLE sales
```

```

ADD PROJECTION sales_projection (
  SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
  GROUP BY product_id
);

ALTER TABLE sales
  (ADD PROJECTION sales_projection
  (
    SELECT
      product_id,
      sum(quantity) AS total_quantity,
      sum(quantity * price) AS total_sales
    GROUP BY product_id
  ))

```

Query id: b93f9497-1098-4bd7-a95c-0e5e70067fca

Ok.

0 rows in set. Elapsed: 0.014 sec.

```
ALTER TABLE sales MATERIALIZE PROJECTION sales_projection;
```

```
ALTER TABLE sales
  (MATERIALIZE PROJECTION sales_projection)
```

Query id: 412c8855-e1d5-43fa-a4ef-993fd43661c1

Ok.

0 rows in set. Elapsed: 0.020 sec.

```

SELECT
  product_id,
  sum(quantity) AS total_quantity,
  sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id
SETTINGS force_optimize_projection_name = 'sales_projection';

SELECT
  product_id,
  sum(quantity) AS total_quantity,
  sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id
SETTINGS force_optimize_projection_name = 'sales_projection'

```

Query id: 16952c1b-ed27-4b9c-a1fb-7a190c547a7c

➤ Progress: 1.00 thousand rows, 36.00 KB (253.76 thousand rows/s., 9.14 MB/s.) → Progress: 1.00 thousand rows, 36.00 KB (253.76 thousand rows/s., 9.14 MB/s.)

1000 rows in set. Elapsed: 0.004 sec. Processed 1.00 thousand rows, 36.00 KB (245.24 thousand rows/s., 8.83 MB/s.)
Peak memory usage: 76.66 KiB.

При стандартном запросе

```

SELECT
  product_id,
  sum(quantity) AS total_quantity,
  sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id

```

```

SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id

```

Query id: 4281dd0f-29fe-4657-abef-d17b3e131698

↓ Progress: 1.00 thousand rows, 36.00 KB (281.36 thousand rows/s., 10.13 MB/s.) ✓ Progress: 1.00 thousand rows, 36.00 KB (281.36 thousand rows/s., 10.13 MB/s.)

1000 rows in set. Elapsed: 0.004 sec. Processed 1.00 thousand rows, 36.00 KB (272.43 thousand rows/s., 9.81 MB/s.)

Peak memory usage: 76.35 KiB.

3. Создаем материальное представление

Создаем таблицу источник для материализационного представления

```

CREATE TABLE sales_mv_table (
    product_id UInt32,
    total_quantity UInt64,
    total_sales Float64
) ENGINE = AggregatingMergeTree()
ORDER BY product_id;

```

```

CREATE TABLE sales_mv_table
(
    `product_id` UInt32,
    `total_quantity` UInt64,
    `total_sales` Float64
)
ENGINE = AggregatingMergeTree
ORDER BY product_id

```

Query id: 02208e20-266d-44fe-9ca9-91a6551bc715

Ok.

0 rows in set. Elapsed: 0.012 sec.

```

CREATE MATERIALIZED VIEW sales_mv
TO sales_mv_table
AS
SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id;

```

```

CREATE MATERIALIZED VIEW sales_mv TO sales_mv_table
AS SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id

```

Query id: fb4cbeaa-23d9-4006-a19b-9a4bbd2a2193

Ok.

0 rows in set. Elapsed: 0.014 sec.

```

INSERT INTO sales_mv_table

```

```

SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id;

```

```

INSERT INTO sales_mv_table SELECT
    product_id,
    sum(quantity) AS total_quantity,
    sum(quantity * price) AS total_sales
FROM sales
GROUP BY product_id

```

Query id: b5903b97-f0cb-4bb2-825a-31fbb9c684a5

↖ Progress: 1.00 thousand rows, 36.00 KB (118.04 thousand rows/s., 4.25 MB/s.) ↑ Progress: 1.00 thousand rows, 36.00 KB (118.04 thousand rows/s., 4.25 MB/s.) 0k.

0 rows in set. Elapsed: 0.008 sec. Processed 1.00 thousand rows, 36.00 KB (118.04 thousand rows/s., 4.25 MB/s.)
Peak memory usage: 3.28 MiB.

```

SELECT
    product_id,
    total_quantity,
    total_sales
FROM sales_mv_table
ORDER BY product_id;

```

```

SELECT
    product_id,
    total_quantity,
    total_sales
FROM sales_mv_table
ORDER BY product_id ASC

```

Query id: efc00582-58dc-4ae1-835a-558661b9026a

→ Progress: 1.00 thousand rows, 20.00 KB (370.06 thousand rows/s., 7.40 MB/s.) ↘ Progress: 1.00 thousand rows, 20.00 KB (370.06 thousand rows/s., 7.40 MB/s.)

1000 rows in set. Elapsed: 0.003 sec. Processed 1.00 thousand rows, 20.00 KB (346.40 thousand rows/s., 6.93 MB/s.)
Peak memory usage: 25.72 KiB.

Итог:

Материализованное представление работает быстрее проекции, а она быстрее чем прямой запрос